```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.api import VAR
from statsmodels.tsa.vector_ar.irf import IRAnalysis
from sklearn.decomposition import FastICA
import math

# Set a consistent style for plots
sns.set(style='whitegrid', rc={'figure.figsize': (10, 6)})
plt.rcParams['figure.dpi'] = 100

def run_6var_ica_vecm_analysis_plot_point_estimates():
    """
    Runs the full ICA-VECM (VAR in Levels) analysis on the 6 core
variables,
    using BIC for optimal lag selection.
    This version MANUALLY PLOTS ONLY THE POINT ESTIMATES for IRFs,
    computes FEVD manually, and includes VAR model summary with
coefficients.
    """
    try:
        # --- Step 1: Load and Clean Data ---
        print("--- Step 1: Loading and Cleaning Data (6 Variables)
---")
        file_name =
"/Users/sarthak/Desktop/TSF_Project/data/final_data_compiled.csv"
        df = pd.read_csv(file_name)
        var_names = [
            'YoY Growth Rate (%)', 'Inflation_CPI', 'WACMR_IR',
            'GBYR_value', 'PLR quarterly', 'NER'
        ]
        df_levels = df[var_names]
        # Check for missing values
        if df_levels.isna().any().any():
            raise ValueError("Data contains missing values. Please
clean the dataset.")
        print(f"Model variables: {var_names}")
        print("\n" + "="*80 + "\n")

        # --- Step 2: Select Optimal Lag Order (BIC) ---
        print("--- Step 2: Selecting Optimal Lag Order (BIC) ---")
        model_for_lag_selection = VAR(df_levels)
        maxlags = 4
        lag_order_results =
model_for_lag_selection.select_order(maxlags=maxlags)
        print(lag_order_results.summary())
        chosen_lag = lag_order_results.bic
        print(f"\nBIC selected p = {chosen_lag} lags.")
```

```
        print("\n" + "="*80 + "\n")

        # --- Step 3: Fit VAR(p) in Levels and Display Summary ---
        print(f"--- Step 3: Fitting VAR({chosen_lag}) Model on Data
Levels ---")
        model = VAR(df_levels)
        model_fit = model.fit(maxlags=chosen_lag)
        print("VAR model fit complete.")
        print(f"Number of observations used: {model_fit.nobs}")
        print("\nVAR Model Summary (Coefficients and Statistics):")
        print(model_fit.summary())
        print("\n" + "="*80 + "\n")

        # --- Step 4: Extract Residuals ---
        print("--- Step 4: Extracting Model Residuals ---")
        residuals = model_fit.resid
        print(f"Residuals extracted. Shape: {residuals.shape}")
        print("\n" + "="*80 + "\n")

        # --- Step 5: Run ICA and Identify Shocks ---
        print("--- Step 5: Running FastICA and Identifying Shocks
---")
        ica = FastICA(n_components=6, whiten='unit-variance',
random_state=42, max_iter=1000)
        ica.fit(residuals)
        print(f"ICA converged in {ica.n_iter_} iterations")
        W = ica.mixing_
        print(f"ICA mixing matrix shape: {W.shape}")

        wacmr_row_index = var_names.index('WACMR_IR')
        policy_shock_index =
int(np.argmax(np.abs(W[wacmr_row_index, :])))
        print(f"Identified Policy Shock: Column {policy_shock_index}
(Structural_Shock_{policy_shock_index+1})")

        if W[wacmr_row_index, policy_shock_index] < 0:
            print("Normalizing sign: Flipping shock column.")
            W[:, policy_shock_index] *= -1

        df_impact = pd.DataFrame(W, index=var_names,
columns=[f'Shock_{i+1}' for i in range(6)])
        print("\nContemporaneous Impact Matrix (ICA Weights /
A_0_inv):")
        print(df_impact)
        print("\n" + "="*80 + "\n")

        # Plot the heatmap
        plt.figure(figsize=(8, 6))
        sns.heatmap(df_impact, annot=True, fmt=".2f",
cmap="coolwarm_r", center=0)
```

```
        plt.title('6-Variable ICA Impact Matrix (SIRF at Lag 0)',
fontsize=16)
        plt.xlabel('Identified Structural Shocks', fontsize=12)
        plt.ylabel('Variables (in Levels)', fontsize=12)
        plt.tight_layout()
        plt.savefig('ica_6var_heatmap.png')
        print("Saved heatmap to 'ica_6var_heatmap.png'\n")
        plt.clf()

        # --- Step 6: Generate and Manually Plot Structural IRFs
(Point Estimates Only) ---
        print("--- Step 6: Generating & Manually Plotting Dynamic
Structural IRFs (Point Estimates Only) ---")

        periods = 20

        # Initialize IRAnalysis *without* bootstrap arguments
        irf = IRAnalysis(model_fit, P=W, periods=periods)
        # Debug: Check available attributes
        print("IRAnalysis attributes:", dir(irf))

        # Get ONLY the point estimates
        irf_point_estimates = irf.irfs
        print(f"IRF point estimates shape:
{irf_point_estimates.shape}")

        # Create subplots
        num_vars = len(var_names)
        ncols = 2
        nrows = math.ceil(num_vars / ncols)
        fig, axes = plt.subplots(nrows=nrows, ncols=ncols,
figsize=(ncols*6, nrows*4))
        axes = axes.flatten()

        for i, var_name in enumerate(var_names):
            ax = axes[i]

            # Extract IRF point estimate for this variable responding
to the policy shock
            response_series = irf_point_estimates[:, i,
policy_shock_index]

            # Plot ONLY the point estimate
            ax.plot(range(periods + 1), response_series, color='blue',
label='Point Estimate')
            ax.axhline(0, color='grey', linestyle='--', linewidth=0.8)
            ax.set_title(f'Response of {var_name}')
            ax.set_xlabel('Quarters Ahead')
            ax.set_ylabel('Impact')
            ax.grid(True, linestyle=':', alpha=0.6)
```

```
            if i == 0: ax.legend()

        # Remove any unused subplots
        if num_vars < len(axes):
            for i in range(num_vars, len(axes)):
                fig.delaxes(axes[i])

        fig.suptitle(f'Response to Monetary Policy Shock (Identified
as Shock {policy_shock_index+1}) - Point Estimates', fontsize=18,
y=1.03)
        fig.tight_layout(rect=[0, 0.03, 1, 0.98])

        plt.savefig('sirf_ica_6var_point_estimates_plot.png')
        print(f"Successfully generated and saved POINT ESTIMATE IRF
plot to 'sirf_ica_6var_point_estimates_plot.png'\n")

        # --- Step 7: Generate Forecast Error Variance Decomposition
(FEVD) ---
        print("--- Step 7: Generating Forecast Error Variance
Decomposition (FEVD) ---")

        # Attempt to compute FEVD
        try:
            fevd = irf.fevd(periods=20)
            print("\nFEVD Summary (Full):")
            print(fevd.summary())

            print("\n--- FEVD Highlights: % Variance Explained by
Policy Shock ---")
            for i in [1, 4, 8, 12, 20]:
                print(f"  At {i} quarters:")
                fevd_results = fevd.decomp[i-1]
                policy_shock_variance =
fevd_results[policy_shock_index]

                for j, var in enumerate(var_names):
                    print(f"    - {var}:
{policy_shock_variance[j]*100:.2f}%")
        except AttributeError:
            print("WARNING: FEVD computation failed. Computing manual
FEVD.")
            # Manual FEVD calculation
            irfs = irf.irfs
            n_vars = len(var_names)
            fevd_manual = np.zeros((periods, n_vars, n_vars))

            for h in range(periods):
                # Compute cumulative IRF squared for each horizon
                irf_squared = np.square(irfs[:h+1])
                # Sum over time to get total variance contribution
```

```
                total_variance = np.sum(irf_squared, axis=0)
                # Normalize to get proportion of variance
                total_variance_sum = np.sum(total_variance, axis=1,
keepdims=True)
                fevd_manual[h] = total_variance / (total_variance_sum
+ 1e-10)

            print("\n--- Manual FEVD Highlights: % Variance Explained
by Policy Shock ---")
            for i in [1, 4, 8, 12, 20]:
                print(f"  At {i} quarters:")
                policy_shock_variance = fevd_manual[i-1, :,
policy_shock_index]
                for j, var in enumerate(var_names):
                    print(f"    - {var}:
{policy_shock_variance[j]*100:.2f}%")

        print("\n" + "="*80 + "\n")
        print("ANALYSIS COMPLETE.")

    except FileNotFoundError:
        print(f"ERROR: File not found. Make sure '{file_name}' is
available.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

# Run the entire analysis pipeline
run_6var_ica_vecm_analysis_plot_point_estimates()
```

```
--- Step 1: Loading and Cleaning Data (6 Variables) ---
Model variables: ['YoY Growth Rate (%)', 'Inflation_CPI', 'WACMR_IR',
'GBYR_value', 'PLR quarterly', 'NER']

================================================================================
==========

--- Step 2: Selecting Optimal Lag Order (BIC) ---
 VAR Order Selection (* highlights the minimums)
=================================================
      AIC         BIC         FPE         HQIC
-------------------------------------------------
0       8.089       8.271       3260.       8.162
1     0.2161*      1.485*      1.245*     0.7241*
2      0.4339       2.791       1.573       1.377
3       1.029       4.474       2.972       2.408
4      0.5399       5.072       1.975       2.354
-------------------------------------------------

BIC selected p = 1 lags.
```

```
==========

--- Step 3: Fitting VAR(1) Model on Data Levels ---
VAR model fit complete.
Number of observations used: 81

VAR Model Summary (Coefficients and Statistics):
  Summary of Regression Results
==================================
Model:                         VAR
Method:                        OLS
Date:           Sat, 18, Oct, 2025
Time:                      15:54:10
--------------------------------------------------------------------
No. of Equations:         6.00000    BIC:                    1.32654
Nobs:                     81.0000    HQIC:                  0.583109
Log likelihood:          -651.046    FPE:                    1.09152
AIC:                    0.0849763    Det(Omega_mle):        0.663812
--------------------------------------------------------------------
Results for equation YoY Growth Rate (%)
==========================================================================
====================
                            coefficient       std. error           t-
stat            prob
--------------------------------------------------------------------------
--------------------
const                         20.078805         9.426091
2.130           0.033
L1.YoY Growth Rate (%)         0.417055         0.107182
3.891           0.000
L1.Inflation_CPI              -0.032442         0.362568           -
0.089           0.929
L1.WACMR_IR                    0.190483         0.465636
0.409           0.682
L1.GBYR_value                 -0.181716         0.636065           -
0.286           0.775
L1.PLR quarterly              -0.794562         0.554632           -
1.433           0.152
L1.NER                        -0.140805         0.070209           -
2.005           0.045
==========================================================================
====================

Results for equation Inflation_CPI
==========================================================================
====================
                            coefficient       std. error           t-
stat            prob
--------------------------------------------------------------------------
```

```
-------------------
const                          1.647643         3.061756
0.538           0.590
L1.YoY Growth Rate (%)        -0.032314         0.034814           -
0.928           0.353
L1.Inflation_CPI               0.073894         0.117768
0.627           0.530
L1.WACMR_IR                    0.027557         0.151247
0.182           0.855
L1.GBYR_value                  0.231324         0.206605
1.120           0.263
L1.PLR quarterly              -0.053685         0.180154           -
0.298           0.766
L1.NER                        -0.023263         0.022805           -
1.020           0.308
================================================================================
====================

Results for equation WACMR_IR
================================================================================
====================
                            coefficient       std. error           t-
stat            prob
--------------------------------------------------------------------------------
-------------------
const                          0.927289         1.102480
0.841           0.400
L1.YoY Growth Rate (%)        -0.005207         0.012536           -
0.415           0.678
L1.Inflation_CPI               0.000757         0.042406
0.018           0.986
L1.WACMR_IR                    0.900833         0.054461
16.541           0.000
L1.GBYR_value                  0.121283         0.074394
1.630           0.103
L1.PLR quarterly              -0.055869         0.064870           -
0.861           0.389
L1.NER                        -0.010785         0.008212           -
1.313           0.189
================================================================================
====================

Results for equation GBYR_value
================================================================================
====================
                            coefficient       std. error           t-
stat            prob
--------------------------------------------------------------------------------
-------------------
```

```
const                            3.202695         0.894943
3.579           0.000
L1.YoY Growth Rate (%)           0.011541         0.010176
1.134           0.257
L1.Inflation_CPI                -0.011044         0.034423           -
0.321           0.748
L1.WACMR_IR                      0.051231         0.044209
1.159           0.247
L1.GBYR_value                    0.834088         0.060390
13.812           0.000
L1.PLR quarterly                -0.142445         0.052658           -
2.705           0.007
L1.NER                          -0.017195         0.006666           -
2.580           0.010
================================================================================
====================

Results for equation PLR quarterly
================================================================================
====================
                            coefficient       std. error           t-
stat            prob
--------------------------------------------------------------------------------
--------------------
const                            2.530116         1.142313
2.215           0.027
L1.YoY Growth Rate (%)           0.000917         0.012989
0.071           0.944
L1.Inflation_CPI                 0.007691         0.043938
0.175           0.861
L1.WACMR_IR                     -0.014357         0.056429           -
0.254           0.799
L1.GBYR_value                    0.082366         0.077082
1.069           0.285
L1.PLR quarterly                 0.801264         0.067214
11.921           0.000
L1.NER                          -0.019074         0.008508           -
2.242           0.025
================================================================================
====================

Results for equation NER
================================================================================
====================
                            coefficient       std. error           t-
stat            prob
--------------------------------------------------------------------------------
--------------------
const                           -3.418934         3.591670           -
```

```
0.952           0.341
L1.YoY Growth Rate (%)          0.015158         0.040840
0.371           0.711
L1.Inflation_CPI                0.131227         0.138151
0.950           0.342
L1.WACMR_IR                     0.055154         0.177424
0.311           0.756
L1.GBYR_value                   0.296729         0.242363
1.224           0.221
L1.PLR quarterly                0.010025         0.211334
0.047           0.962
L1.NER                          1.014280         0.026752
37.914           0.000
==============================================================================
====================

Correlation matrix of residuals
                      YoY Growth Rate (%)  Inflation_CPI  WACMR_IR
GBYR_value  PLR quarterly       NER
YoY Growth Rate (%)              1.000000       0.061216  0.132699
0.148884       0.104244 -0.274811
Inflation_CPI                    0.061216       1.000000 -0.059407
0.275647      -0.076916  0.167249
WACMR_IR                         0.132699      -0.059407  1.000000
0.150541      -0.010577  0.079880
GBYR_value                       0.148884       0.275647  0.150541
1.000000       0.146874  0.009784
PLR quarterly                    0.104244      -0.076916 -0.010577
0.146874       1.000000 -0.127440
NER                             -0.274811       0.167249  0.079880
0.009784      -0.127440  1.000000




==============================================================================
==========

--- Step 4: Extracting Model Residuals ---
Residuals extracted. Shape: (81, 6)

==============================================================================
==========

--- Step 5: Running FastICA and Identifying Shocks ---
ICA converged in 12 iterations
ICA mixing matrix shape: (6, 6)
Identified Policy Shock: Column 5 (Structural_Shock_6)
Normalizing sign: Flipping shock column.
```

```
Contemporaneous Impact Matrix (ICA Weights / A_0_inv):
                      Shock_1   Shock_2   Shock_3   Shock_4   Shock_5
\
YoY Growth Rate (%)  0.066819  4.011734  0.497090  0.111920 -0.063873

Inflation_CPI         0.261553  0.071381  0.247857 -0.910861  0.841892

WACMR_IR             -0.000569  0.088892  0.009045  0.031028  0.109092

GBYR_value            0.011710  0.020325  0.295392  0.127748  0.209195

PLR quarterly        -0.478381  0.048405  0.086799 -0.012676  0.038802

NER                   0.084689 -0.313514 -0.930840  0.420278  1.107723


                      Shock_6
YoY Growth Rate (%) -0.227813
Inflation_CPI       -0.241656
WACMR_IR             0.451323
GBYR_value          -0.008459
PLR quarterly       -0.025836
NER                 -0.086666

==========================================================================
==========

Saved heatmap to 'ica_6var_heatmap.png'

--- Step 6: Generating & Manually Plotting Dynamic Structural IRFs
(Point Estimates Only) ---
IRAnalysis attributes: ['G', 'H', 'P', 'T', '_A', '__class__',
'__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__firstlineno__', '__format__', '__ge__', '__getattribute__',
'__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__',
'__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__static_attributes__', '__str__', '__subclasshook__', '__weakref__',
'_choose_irfs', '_eigval_decomp_SZ', '_empty_covm', '_g_memo',
'_orth_cov', 'cov', 'cov_a', 'cov_sig', 'cum_effect_cov',
'cum_effect_stderr', 'cum_effects', 'cum_errband_mc', 'err_band_sz1',
'err_band_sz2', 'err_band_sz3', 'errband_mc', 'fevd_table', 'irfs',
'lags', 'lr_effect_cov', 'lr_effect_stderr', 'lr_effects', 'model',
'neqs', 'order', 'orth_cum_effects', 'orth_irfs', 'orth_lr_effects',
'periods', 'plot', 'plot_cum_effects', 'stderr', 'svar']
IRF point estimates shape: (21, 6, 6)
Successfully generated and saved POINT ESTIMATE IRF plot to
'sirf_ica_6var_point_estimates_plot.png'

--- Step 7: Generating Forecast Error Variance Decomposition (FEVD)
```

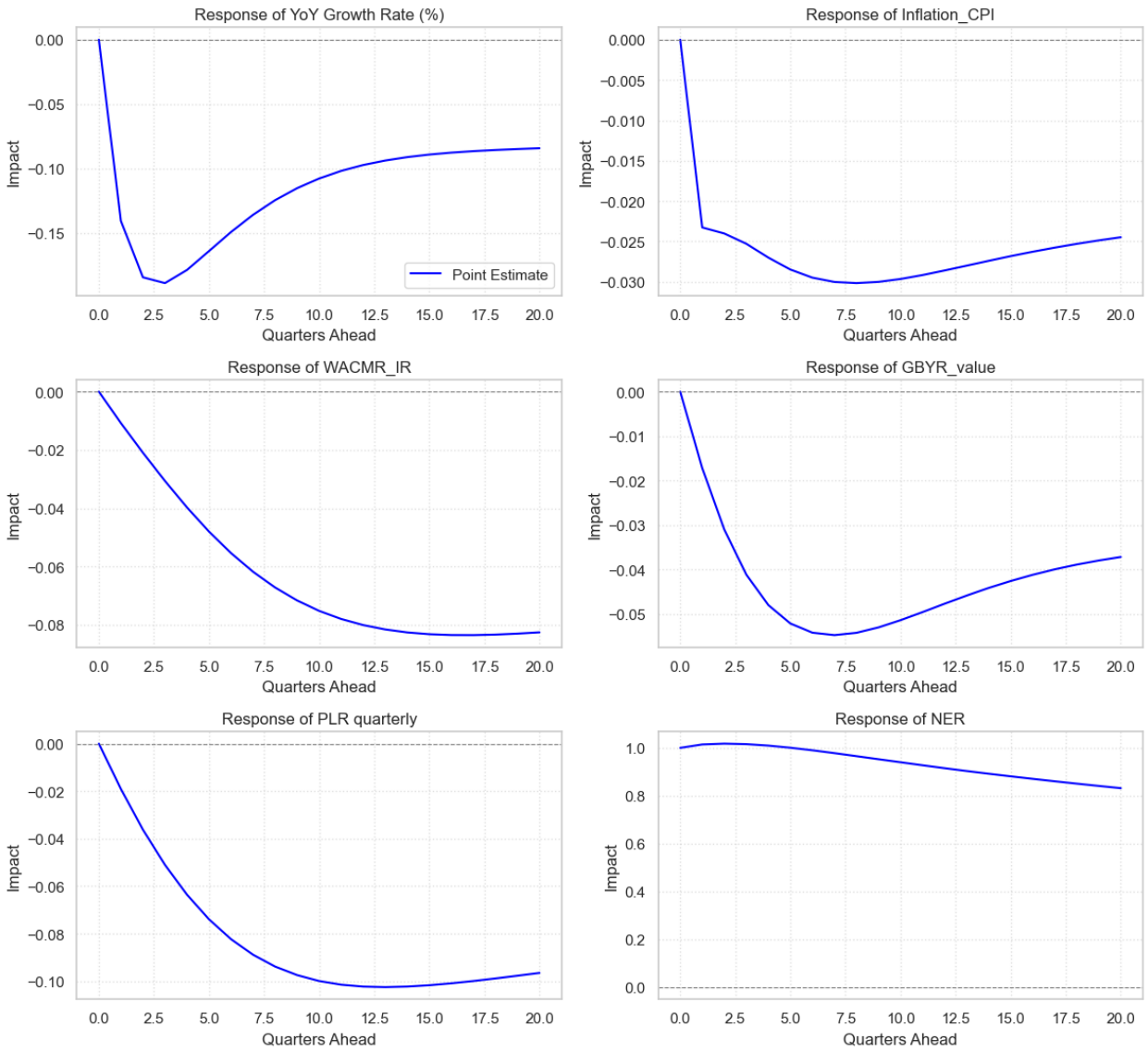
```
---
WARNING: FEVD computation failed. Computing manual FEVD.

--- Manual FEVD Highlights: % Variance Explained by Policy Shock ---
  At 1 quarters:
    - YoY Growth Rate (%): 0.00%
    - Inflation_CPI: 0.00%
    - WACMR_IR: 0.00%
    - GBYR_value: 0.00%
    - PLR quarterly: 0.00%
    - NER: 100.00%
  At 4 quarters:
    - YoY Growth Rate (%): 2.22%
    - Inflation_CPI: 0.15%
    - WACMR_IR: 0.05%
    - GBYR_value: 0.11%
    - PLR quarterly: 0.18%
    - NER: 76.52%
  At 8 quarters:
    - YoY Growth Rate (%): 3.13%
    - Inflation_CPI: 0.42%
    - WACMR_IR: 0.23%
    - GBYR_value: 0.39%
    - PLR quarterly: 1.06%
    - NER: 45.94%
  At 12 quarters:
    - YoY Growth Rate (%): 3.56%
    - Inflation_CPI: 0.69%
    - WACMR_IR: 0.51%
    - GBYR_value: 0.63%
    - PLR quarterly: 2.44%
    - NER: 30.54%
  At 20 quarters:
    - YoY Growth Rate (%): 3.95%
    - Inflation_CPI: 1.13%
    - WACMR_IR: 1.16%
    - GBYR_value: 0.96%
    - PLR quarterly: 4.85%
    - NER: 18.16%

================================================================================
==========

ANALYSIS COMPLETE.

<Figure size 800x600 with 0 Axes>
```




```python
import matplotlib.pyplot as plt
import pandas as pd

# FEVD data
fevd_data = {
    "Horizon": [1, 4, 8, 12, 20],
    "YoY Growth Rate (%)": [0.00, 2.22, 3.13, 3.56, 3.95],
    "Inflation_CPI": [0.00, 0.15, 0.42, 0.69, 1.13],
    "WACMR_IR": [0.00, 0.05, 0.23, 0.51, 1.16],
    "GBYR_value": [0.00, 0.11, 0.39, 0.63, 0.96],
    "PLR quarterly": [0.00, 0.18, 1.06, 2.44, 4.85],
    "NER": [100.00, 76.52, 45.94, 30.54, 18.16]
```

```
}

df_fevd = pd.DataFrame(fevd_data)

# Plot
plt.figure(figsize=(10,6))
for col in df_fevd.columns[1:]:
    plt.plot(df_fevd["Horizon"], df_fevd[col], marker='o', label=col)

plt.title("Forecast Error Variance Decomposition (FEVD) – Policy Shock
Impact")
plt.xlabel("Horizon (Quarters Ahead)")
plt.ylabel("% Variance Explained by Policy Shock")
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.savefig("fevd_policy_shock.png", dpi=300)
plt.show()
```